Exact State, Partial Observation, and Reversible Recovery

A reproducible synthetic test environment

TASUMER MAF | Technical report 0.1.0 | 25 October 2026

Release candidate. This is a methods and software report, not an externally peer-reviewed paper. Publication licensing remains an owner decision.

Abstract

A visible reading can recur while the state that produces it has changed. This report specifies a small exact-arithmetic environment that separates a numerical field, an ordered observation window, a carried reference, and an instruction history. It implements additive changes, reader shifts and passive coordinate changes, together with checkpoint restoration and journal replay. The public experiment uses independently generated rational fields on 81 addresses and a registered suite of 512 forty-step journals. Checkpoint recovery, replay from a known initial state and inverse return all succeed in 512 of 512 cases. A separately implemented coordinate-dictionary oracle agrees at all 20,480 trace steps and 3,402 primitive cases. Controls retain an observation collision, a history collision, and one accidental endpoint agreement after instruction-order reversal. These results establish a reproducible deterministic software capability. They neither measure language-model performance nor imply that checkpointing or reversible computation is a new concept.

1. The question being tested

Suppose a program reads nine selected entries from a larger field. It changes the field, changes where it looks, and later produces those nine values again. Has it returned? There are at least three different questions:

1. Does the visible reading agree?

2. Do the full field, ordered reader and carried reference agree?

3. Was the same route taken?

Agreement on the first question does not answer the second. Agreement on the second does not identify the third. The environment makes these distinctions explicit so that a tool, learner or future agent benchmark can test the claim actually being made.

The contribution is an inspectable specification, implementation and test environment. Reversible computation has a long history: Bennett's construction already uses retained intermediate information and backward execution [1]. Complete action histories and replay also belong to established event-sourcing practice [2]. We do not claim precedence over either. The present implementation is deliberately small enough to compare with an independent oracle.

2. State and observation

Let X be a finite set of N indexed addresses. A state is

```
s = (h, r, o)
h : X -> Q
r = (r1, ..., rm), with distinct ri in X
o in X
```

Here h is an exact rational field, r is an ordered reader, and o is a carried reference address. The observation is

```
O(s) = (h(r1), ..., h(rm)).
```

The reader is not an instruction history. It says where and in what order the current state is read. The reference is another component of the state, even when it does not affect this particular observation function.

For unrestricted fields with fixed r and o, observation has a direct ambiguity: altering an unsampled address leaves O(s) unchanged. When m < N, this gives N - m independent hidden value directions in the full rational vector space. This statement does not assert that every such direction belongs to a narrower family imposed by additional constraints.

The public collision control starts with a generated field and adds one at a declared unsampled address. It therefore tests the full engine's state space, not a claim that the perturbed field remains in the initial balanced fixture family. A predeclared reader shift subsequently exposes that address.

3. Operations and exact inverse

Three operation types are sufficient for this test.

3.1 Add a field

For a fixed rational field a and integer count k:

```
(h, r, o) -> (h + k*a, r, o).
```

The inverse changes k to -k. The uniform field -1 supplies the mean-lowering operation. Other fields may have zero mean without changing the average.

3.2 Move the reader

For a permutation pi of X:

```
(h, r, o) -> (h, pi^k(r), o).
```

The permutation acts on each entry of the ordered reader. Its inverse is pi^(-k). A fixed additive field and a reader permutation can be combined in one operation because they affect different state components. In this model the addition is defined in fixed address coordinates, independently of r.

3.3 Change coordinates passively

Move the field, reader and reference together:

```
h'(pi^k(x)) = h(x)
r' = pi^k(r)
o' = pi^k(o).
```

Every observed value remains unchanged, since h'(pi^k(ri)) = h(ri). This is an exact statement about a relabeling operation, not evidence that no other state component changed. Its inverse again negates k.

An additive change and passive reframe generally do not commute. The engine therefore rejects a single operation definition that tries to combine both. They can be composed as separate journal entries, whose inverse order is clear.

3.4 Inverting a journal

For an ordered journal J = [(g1,k1), ..., (gt,kt)], define

```
J^-1 = [(gt,-kt), ..., (g1,-k1)].
```

Each primitive cancels its partner. Applying J and then J^-1 restores all components of s, by successive cancellation. This proof covers any finite journal using the allowed operation types, not just the tested sequences. The software checks whether the implementation obeys that specification.

Different journals can still have the same endpoint. For example, adding a field and lowering the mean commute. Their two possible orders are distinct histories with equal final states. No endpoint-only procedure can distinguish such histories without additional recorded information.

4. The public synthetic fixture

The address set is X = {-1,0,+1}^4, in lexicographic order. It contains 3^4 = 81 addresses. Define w(k) = ((k+1) modulo 3) - 1. Thus successive labels cycle through -1, 0, +1, then -1 again. This is a finite address convention, not an inference about a physical dimension.

The six instruction names have the following public definitions. These fields were generated for this release; they are not the private numerical fields with similar names in the originating study.

Name	Change to values	Change to addresses
A	Add w(x+y+z+t)	None
B	Add w(x-y+z-t)	Move reader by u=(0,1,1,0)
M	Subtract 1 at every address	None
R	Passive reframe	(x,y,z,t) -> (z,y,t,x)
U	Passive reframe	Translate by u=(0,1,1,0)
S	None	Move reader by q=(-1,-1,1,1)

Translation wraps each address label with w. R and U have order three. A count k repeats an operation algebraically; negative k invokes its inverse. The core engine itself accepts any finite permutation, including the four-cycle covered by a unit test. It is not restricted to order-three address changes.

Sixteen initial fields are generated without a source dataset. For fixture i = 0,...,15, let mu_i = 11 + i/5. Enumerate the eight directions d_j = (1, +/-1, +/-1, +/-1) lexicographically. Set

```
c_ij = (((i+2)*(j+3) modulo 11) - 5) / ((j modulo 3) + 1)
h_i(p) = mu_i + sum_j c_ij * w(d_j dot p).
```

All coefficients are exact rationals. On any line obtained by varying one coordinate through -1, 0, +1, each wave takes all three labels and sums to zero. Every such line in h_i therefore sums to 3*mu_i. This balance follows from the generator; it is not a measured discovery or an assertion about every diagonal.

The initial reader is the ordered list (a,b,0,w(a+b)) for a,b in {-1,0,+1}. The 512 trial references cycle through all 81 addresses. The primitive checks separately test every reference. All formulas and indexing conventions appear in `src/crossroads_state/synthetic.py`.

5. Checkpoint and journal contracts

The checkpoint payload contains a schema identifier, a model fingerprint and the full state. Rational entries are serialized as strings such as "-2/3"; floating-point approximations are not accepted. The reader order and reference index are serialized explicitly. The model fingerprint is a SHA-256 digest of the declared dimensions, name and complete operation definitions.

A SHA-256 digest covers the canonical JSON payload. A checkpoint for another model is rejected. An array mutation, reader swap or reference change fails verification when the original digest is retained. However, the digest is unkeyed. Recomputing it after editing the payload creates a different valid checkpoint. The included test requires this outcome; no authentication claim is made. NIST specifies SHA-256 as part of its Secure Hash Standard [3].

Checkpoint recovery needs the complete checkpoint and matching model. Journal replay needs a known initial state, the full ordered journal and matching instruction definitions. Neither method has an informational advantage when the other is supplied equally complete information. We report both as exact baselines rather than omit replay to create a false comparison.

This library has no distributed processes, clocks or messages in flight. It does not solve the consistent distributed snapshot problem addressed by Chandy and Lamport [4]. It operates on a single already defined state in one process. Durable database storage, concurrent updates and crash-safe writes remain engineering work outside this implementation.

6. Registered protocol and independent oracle

`registration.json` was written before the implementation was executed. It fixes seed 20261025, 512 trials, 40 instructions per trial, 16 initial fixtures, six operation names and counts drawn from {-2,-1,+1,+2}. It also fixes primitive counts {-3,-2,-1,0,+1,+2,+3} and all 81 reference positions. The trial journals are generated independently of the destination states.

The oracle represents the field as a dictionary keyed by coordinate tuples. It implements each synthetic instruction directly, with an independent wrapping function and coordinate formulas. It does not call the production engine, permutation helper or fixture generator to obtain expected transformations. Both implementations receive the same initial field and stated operation specification. This is useful implementation independence; it is not an independent derivation of the specification or external peer review.

At every trace step, all 81 values, all nine ordered reader addresses and the reference must agree exactly. The primitive audit has 81 references x 6 operations x 7 counts = 3,402 cases. Each case is also followed by its recorded inverse. All comparisons use rational equality, with no numerical tolerance.

The controls remove the final instruction, reverse journal order, independently shuffle journal order, mutate checkpoint components, and deliberately create equal observations from unequal fields. The corrupted-journal controls are descriptive: accidental endpoint agreements are counted and retained. They are not assumed impossible.

7. Results

Measure	Exact result
Checkpoint recovery	512 / 512
Known-initial-state journal replay	512 / 512
Inverse return of full state	512 / 512
Independent oracle trace-step agreement	20,480 / 20,480
Independent oracle primitive agreement	3,402 / 3,402
Primitive inverse return	3,402 / 3,402
Changed-value checksum rejection	512 / 512
Changed-reader checksum rejection	512 / 512
Changed-reference checksum rejection	512 / 512
Endpoint changed after truncation	512 / 512
Endpoint changed after reversed order	511 / 512
Endpoint changed after shuffled order	512 / 512

The observation-collision control has equal initial readings, unequal full states, and unequal readings after the declared S shift. The commuting-history control has distinct journals and the same endpoint. An edited state with a recomputed digest is accepted, as required by the stated checksum limitation.

One reversed-order journal reaches the same endpoint as the original. Retaining this result matters: a claim that all edited histories must be detected through endpoint comparison would be false. A separate digest over an original journal can detect changes to its bytes when retained independently, but it would not establish that the original journal was true.

Canonical checkpoint payloads in this suite range from 729 to 1,003 bytes. Their exact mean is 479511/512 bytes, approximately 936.54 bytes. This is a format measurement, not an optimized compression result or a comparison against other encodings.

8. Reproduction and interpretation

The runtime, unit tests and registered study use Python's standard library. From the repository root, run:

```
PYTHONPATH=src python3 -m unittest discover -s tests -v
python3 scripts/reproduce.py --check
```

The first command runs 17 unit tests. The second recomputes the study and compares the full result dictionary against `results/synthetic-study.json`. That file includes the registration digest, model fingerprint and journal-stream digest. The CI definition repeats these commands; a local pass is not described as a hosted CI pass.

These results can support teaching about observation, coordinates, hidden state, replay, inverse operations and information loss. They can also provide exact ground truth for a future tool-using-agent experiment. For example, an agent could be asked to maintain the state through a held-out journal, then tested on values, reader order, reference and inverse return. Such an experiment would need independently generated tasks, fixed tools, comparable baselines, explicit context and token budgets, model versions, and error analysis. No such LLM experiment has been run here.

The public synthetic study stands on its own. Earlier private validation is neither redistributed nor counted in the totals above. The release does not demonstrate physical transport, special information-preserving properties of a historical artifact, optimized AI memory, or a newly discovered law of computation. Its narrower contribution is fully inspectable: an exact state contract, a reversible instruction model, and tests that separate visible agreement from full recovery and full recovery from route history.

References

[1] Charles H. Bennett. "Logical Reversibility of Computation." IBM Journal of Research and Development 17(6), 525-532 (1973). DOI: 10.1147/rd.176.0525. Original paper: https://worrydream.com/refs/Bennett_1973_-_Logical_Reversibility_of_Computation.pdf

[2] Microsoft Azure Architecture Center. "Event Sourcing pattern." Official engineering documentation, accessed 25 October 2026. https://learn.microsoft.com/en-us/azure/architecture/patterns/event-sourcing

[3] National Institute of Standards and Technology. "Secure Hash Standard (SHS)." FIPS PUB 180-4 (August 2015). DOI: 10.6028/NIST.FIPS.180-4. https://csrc.nist.gov/pubs/fips/180-4/upd1/final

[4] K. Mani Chandy and Leslie Lamport. "Distributed Snapshots: Determining Global States of Distributed Systems." ACM Transactions on Computer Systems 3(1), 63-75 (1985). DOI: 10.1145/214451.214456. Author's page: https://www.microsoft.com/en-us/research/publication/distributed-snapshots-determining-global-states-distributed-system/